



Modelos de Señalización



Los modelos de señalización son las "políticas" que marcarán la forma en la que las entidades concurrentes se mueven entre las colas de los monitores. Definen qué ocurre cuando un proceso dentro del monitor hace `signal()` sobre una variable de condición: ¿quién ejecuta ahora? ¿el que señala o el que ha sido despertado? ¿quién entra primero a la sección crítica?

Explicación

Estos modelos determinan **cómo se mueven los procesos entre las colas del monitor**:

- **cola de exclusión mutua**
- **colas de las variables de condición**

Y dependiendo de la semántica, puede existir incluso **una cola de cortesía**

¿Por qué existen distintos modelos?

Porque **despertar un proceso** (tras un `wait`) no basta:

hay que decidir **quién sigue ejecutando dentro del monitor**.

La teoría clásica (Hoare, Brinch Hansen, Mesa...) propuso distintas políticas para resolver ese momento crítico.

En la práctica:

- Java y C++ solo soportan **SC (Señalar y Continuar)**.
- Los modelos más fuertes (SU, DR, DE) son **imposibles de implementar** en lenguajes sin prioridades reales ni control del planificador.

Tipos

Señalar y Continuar (SC)



El proceso que **hace la señal (`signal`) continúa ejecutando**. El proceso despertado simplemente queda **listo**, pero NO entra al monitor aún.

Esta es la semántica de:

- Java (wait/notify, Condition)
- C++ (condition_variable)
- Python
- C# (Monitor)

Funcionamiento:

1. Proceso A ejecuta `signal(C)`
2. Proceso B (que estaba en `wait`) se desbloquea, **pero NO entra inmediatamente**
3. B pasa a la **cola de exclusión mutua** y compite por el cerrojo
4. Cuando le toque el turno, reevaluará la condición (por eso usamos `while`)

Colas necesarias:

- 1 cola de exclusión mutua
- N colas (una por cada variable de condición)

Total: 1 + N

⚠ Cuando hablamos de “continuar su ejecución”, no es que la entidad concurrente entre en la sección crítica directamente, sino que, si hay algún proceso ya en la sección crítica pasará a la cola de exclusión mutua. Pasará a continuar su ejecución en el punto que toque.

Señalar con Urgencia (SU)



El proceso despertado **toma el monitor inmediatamente**, y el señalizador **se bloquea** en la cola de cortesía.

Es el modelo que mejor refleja la **semántica ideal** de un monitor:

- Si despierto a alguien porque la condición ahora es verdadera,
- **debo garantizar** que sigue siendo verdadera cuando retome el control.

Flujo:

1. Proceso A hace `signal(C)`
2. Proceso B despierta y **entra directamente a ejecutar dentro del monitor**
3. A se mueve a la **cola de cortesía urgente**
4. Cuando B termina o hace `wait`, A vuelve a ejecutar antes que los procesos externos

Colas necesarias:

- Cola de exclusión mutua
- N colas de condición
- Cola de cortesía urgente

Total: N + 2

Retorno Forzado (DR)

No es muy importante dentro de la asignatura.



La llamada a `signal()` **termina inmediatamente** el procedimiento del señalizador. El desbloqueo entra **sin reevaluar** la condición.

Diferencia con SU:

- En SU, el señalizador **se bloquea** en la cola de cortesía.
- En DR, el señalizador **FINALIZA** el procedimiento actual (es expulsado del monitor).

Desbloquear y Esperar (DE)



El proceso señalizador cede el monitor inmediatamente, pero pasa a la **cola del monitor**, no a una cola especial.

No es muy importante dentro de la asignatura.

Semántica usada en:

- Modula-2
- Concurrent Euclid

Diferencia con SU:

- En SU hay **una cola especial** con prioridad.
- En DE el señalizador vuelve a competir como cualquiera.

Resumen visual para examen

Modelo	¿Quién ejecuta tras <code>signal</code> ?	¿Reevalúa condición?	Colas necesarias
SC (Mesa / Java / C++)	El señalizador	✓ Sí → por eso <code>while</code>	1 + N
SU (Hoare)	El despertado	✗ No	2 + N
DR	El despertado (se fuerza retorno)	✗ No	Como SU
DE	El despertado (señalizador va a cola normal)	✗ No	1 + N